INTERMEDIATE LOG ANALYSIS LAB

Splunk + Linux Auth Logs + Zeek Network Logs, on Kali Linux

Lab Summary

You will stand up Splunk Enterprise (free trial license) on a Kali Linux machine, generate a realistic mixed dataset (SSH brute-force attempt, successful pivot login, a web server access log with a scanning/enumeration pattern, and a short packet capture), ingest it into Splunk, and investigate it using SPL (Search Processing Language) the way a SOC analyst would triage an alert. You will also cross-check your Splunk findings against raw CLI log analysis (grep/awk) so you understand what Splunk is doing under the hood.

Objectives

- Install and configure Splunk Enterprise on Kali Linux and ingest multiple log sources.
- Simulate an SSH brute-force attack and a web reconnaissance scan to generate realistic logs.
- Build SPL searches to detect brute-force patterns, successful compromise after failure, and web scanning behavior.
- Correlate authentication logs with network capture data (Zeek) for the same time window.
- Produce a short incident timeline as a final deliverable.

Prerequisites

- Kali Linux (VM or bare metal), 4 GB RAM minimum, 8 GB+ recommended, ~20 GB free disk space.
- Root or sudo access on the Kali box.
- Internet access for package downloads and the Splunk installer.
- Basic comfort with the Linux terminal; no prior Splunk experience required.

Time Budget (3 - 4 hours total)

Step	Task	Approx. Time
1	Environment setup: update Kali, install Splunk, install supporting tools	45 - 60 min
2	Generate attack data: SSH brute-force + successful login + web scan + packet capture	35 - 45 min
3	Ingest logs into Splunk (indexes, sourcetypes, inputs)	20 - 25 min
4	Baseline CLI log analysis (grep/awk) for comparison	20 min
5	SPL investigation: brute force, compromise, web recon, correlation	60 - 75 min
6	Build a simple dashboard + write the incident summary	20 - 30 min
7	Cleanup / stop services	10 min

1. Environment Setup

1.1 Update Kali and install baseline tools

Open a terminal on Kali. Most of these tools ship with Kali by default; the command below makes sure everything is present and current.

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y net-tools apache2-utils hydra tcpdump wireshark \
auditd apache2 openssh-server curl wget jq git
```

If asked whether non-root users may capture packets (dumpcap), choose Yes, then log out and back in once so your user is added to the wireshark group.

1.2 Enable the services you will attack/monitor

```
sudo systemctl enable --now ssh
sudo systemctl enable --now apache2
sudo systemctl enable --now auditd
sudo systemctl status ssh apache2 auditd --no-pager
```

Confirm all three show "active (running)". You are deliberately running SSH and Apache so you have real services to generate real logs against, inside your own isolated lab VM.

1.3 Install Splunk Enterprise (free trial)

Download the Linux .deb package from splunk.com (free account required) to your Kali host, or use wget with a direct download link copied from the Splunk download page. Replace the filename below with the version you actually downloaded.

```
cd ~/Downloads
sudo dpkg -i splunk-*-linux-2.6-amd64.deb
sudo /opt/splunk/bin/splunk start --accept-license --answer-yes --no-prompt \
--seed-passwd 'ChangeMe!2026'
```

Splunk will print a URL once it finishes starting, normally http://127.0.0.1:8000. Log in with the username admin and the password you set above. The free trial license allows up to 500 MB of indexing per day, which is more than enough for this lab.

> **WATCH FOR: Setup stage**
>
> - Splunk Web must be reachable before you continue - if port 8000 refuses connections, run "sudo /opt/splunk/bin/splunk status" and check splunkd.log under /opt/splunk/var/log/splunk/.
>
> - Make sure the VM clock is correct ("timedatectl") - wrong system time will throw off every timestamp correlation you do later.

1.4 Optional: install Zeek for network-level logs

Zeek gives you structured connection and protocol logs instead of raw packets, which is much closer to what you will see in a real SOC. This step is optional but recommended if your time budget allows the full 4 hours.

```
sudo apt install -y zeek
echo 'export PATH=$PATH:/opt/zeek/bin' >> ~/.bashrc && source ~/.bashrc
zeek -v
```

2. Generate the Attack Data

Everything below targets 127.0.0.1 / your own VM only. Do not point these commands at any host you do not own or have written permission to test.

2.1 Create a throwaway low-privilege account to "compromise"

```
sudo useradd -m labuser
echo 'labuser:Summer2026!' | sudo chpasswd
```

This gives you a realistic weak-but-guessable password so the brute force in the next step can eventually succeed, just like a real credential-stuffing scenario.

2.2 Start a packet capture (leave running in its own terminal)

```
sudo tcpdump -i lo -w ~/lab_capture.pcap port 22 or port 80
```

2.3 Simulate an SSH brute-force attack

Build a small password list that does NOT include the real password yet, run Hydra against it so you get a realistic burst of failures, then add the real password and run once more so you get one success right after the failures - this is the pattern you will hunt for in Splunk.

```
printf 'password\n123456\nletmein\nqwerty\nadmin123\n' > ~/wrong_passwords.txt
hydra -l labuser -P ~/wrong_passwords.txt ssh://127.0.0.1 -t 4

echo 'Summer2026!' >> ~/wrong_passwords.txt
hydra -l labuser -P ~/wrong_passwords.txt ssh://127.0.0.1 -t 4
```

2.4 Simulate web reconnaissance / scanning against Apache

```
for path in admin login.php wp-admin phpmyadmin .env config.php backup.zip; do
curl -s -o /dev/null -w "%{http_code} /$path\n" http://127.0.0.1/$path
done

# Generate normal baseline traffic too, so the data is not 100% malicious
for i in 1 2 3 4 5; do curl -s -o /dev/null http://127.0.0.1/; sleep 1; done
```

2.5 Stop the packet capture

Return to the tcpdump terminal and press Ctrl+C. Confirm ~/lab_capture.pcap exists and is non-empty.

> **WATCH FOR: Data-generation stage**
>
> - auth.log should show a tight cluster of "Failed password for labuser" lines followed by one "Accepted password for labuser" - if Hydra fails outright, check that SSH password authentication is enabled in /etc/ssh/sshd_config (PasswordAuthentication yes) and restart ssh.
>
> - Apache access log should show a run of 404s for the scanned paths within a few seconds of each other.

3. Ingest the Logs into Splunk

3.1 Create a dedicated index

In Splunk Web: Settings > Indexes > New Index. Name it lab_logs, leave defaults, click Save.

Or from the CLI:

```
sudo /opt/splunk/bin/splunk add index lab_logs -auth admin:'ChangeMe!2026'
```

3.2 Add the authentication log

Settings > Add Data > Monitor > Files & Directories. Point it at /var/log/auth.log.

- Source type: select linux_secure (if offered) or set sourcetype manually to linux_secure.
- Index: lab_logs.
- Host: leave as the system default.

3.3 Add the Apache access log

Repeat Add Data, this time pointing at /var/log/apache2/access.log.

- Source type: access_combined (Splunk usually auto-detects this).
- Index: lab_logs.

3.4 Add Zeek logs (if you installed Zeek)

Run Zeek against the capture file to produce structured logs, then point Splunk at the resulting directory.

```
mkdir -p ~/zeek_logs && cd ~/zeek_logs
zeek -r ~/lab_capture.pcap local
ls # expect conn.log, ssh.log, http.log, etc.
```

Add Data > Monitor > Files & Directories, point at ~/zeek_logs, sourcetype bro:conn (or zeek:conn), index lab_logs.

3.5 Confirm ingestion

Go to Search, run the following, and confirm you see hits from all three sources:

```
index=lab_logs | stats count by sourcetype
```

4. Baseline CLI Log Analysis (for comparison)

Before leaning on Splunk, confirm you can find the same findings by hand. This builds the habit of understanding what a SIEM query is actually doing, and gives you a sanity check if your Splunk search ever looks wrong.

```
# Count failed vs accepted SSH logins
grep -c 'Failed password' /var/log/auth.log
grep -c 'Accepted password' /var/log/auth.log

# Show the failure-then-success pattern for labuser
grep 'labuser' /var/log/auth.log | tail -20

# Count requests per source path in Apache, sorted
awk '{print $7}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head -15

# Count 404 responses (classic recon signal)
awk '$9==404' /var/log/apache2/access.log | wc -l
```

WATCH FOR: CLI baseline

- Note the exact timestamp of the first failed attempt and the timestamp of the first accepted login for labuser - you will use this time range to scope your Splunk searches.
- Note which paths returned 404 in Apache - this is your indicator list for the web-recon search.

5. SPL Investigation in Splunk

5.1 Find the brute-force cluster

```
index=lab_logs sourcetype=linux_secure "Failed password"
| bin _time span=1m
| stats count by _time, user, src_ip
| where count > 3
```

This buckets failed logins into one-minute windows and surfaces any user/source combination with more than three failures - a classic brute-force threshold.

5.2 Find the successful login that follows the failures

```
index=lab_logs sourcetype=linux_secure ("Failed password" OR "Accepted password")
| transaction user startswith="Failed password" endswith="Accepted password" maxspan=5m
| table _time, user, duration, eventcount
```

The transaction command stitches together a run of failures that ends in a success within a 5-minute window - exactly the sequence you created in Section 2.3.

5.3 Detect the web reconnaissance pattern

```
index=lab_logs sourcetype=access_combined status=404
| stats count, values(uri_path) as paths_tried by clientip
| where count >= 4
```

A single client IP hitting four or more distinct nonexistent paths in a short window is a strong indicator of directory/file enumeration.

5.4 Correlate timing across sources

```
index=lab_logs (sourcetype=linux_secure OR sourcetype=access_combined)
| sort _time
| table _time, sourcetype, src_ip, clientip, user, uri_path, status
```

Sorted side by side, check whether the SSH brute force and the web scan overlap in time - if they came from the same source IP, that strengthens the case that this is one actor doing broad reconnaissance, not two unrelated events.

5.5 (If Zeek is installed) confirm at the network layer

```
index=lab_logs sourcetype="bro:ssh" OR sourcetype="zeek:ssh"
| table ts, id.orig_h, id.resp_h, auth_success, auth_attempts
```

Zeek's own auth_attempts field is a second, independent confirmation of the brute-force count you already found in auth.log - in a real investigation, agreement across two independent log sources is what you cite as strong evidence.

> **WATCH FOR: SPL investigation stage**
>
> - If "transaction" returns nothing, loosen maxspan or confirm your two Hydra runs actually landed within 5 minutes of each other - re-run Section 2.3 if the gap was too large.
>
> - Watch the count of failures per minute, not just the total - a slow, spread-out set of failures reads very differently (and is harder to detect) than a tight burst.

- Note the exact src_ip / clientip values you find - you will need them for the final report.

6. Build a Simple Dashboard and Summarize

6.1 Save key searches as panels

From each search in Section 5, click Save As > Dashboard Panel, and add it to a new dashboard named "Lab - Brute Force and Recon Investigation". Do this for the searches in 5.1, 5.3 and 5.4 at minimum.

6.2 Write a short incident summary

Produce a one-page writeup (plain text or notes file) answering:

1. What was the source IP / user targeted, and over what time window?
2. How many failed authentication attempts occurred before the successful login?
3. Was there a related web reconnaissance pattern from the same source, and if so what paths were probed?
4. What single SPL query would you schedule as an ongoing alert to catch this pattern automatically, and what threshold would you set?

This mirrors the deliverable a junior analyst produces after triaging a real alert: a tight, evidence-backed timeline rather than a raw data dump.

6.3 Stretch goals (if time remains)

- Build a Splunk alert that fires when the 5.1 query returns results, scheduled to run every 5 minutes.
- Add a lookup table mapping src_ip to a fake "asset owner" field and enrich your searches with it.
- Re-run the whole lab with the brute force spread over 20 minutes instead of a tight burst, and see whether your 5.1 query still catches it - adjust the bin span and threshold until it does.

7. Cleanup

```
sudo /opt/splunk/bin/splunk stop
sudo userdel -r labuser
# Optional: remove the Splunk install entirely
# sudo /opt/splunk/bin/splunk stop && sudo rm -rf /opt/splunk
```

If you plan to repeat this lab later with fresh data, keep Splunk installed and simply delete and recreate the lab_logs index instead of removing the whole install.

What You Should Be Able to Explain Afterward

- The difference between raw log review (grep/awk) and SIEM-based review (SPL), and when each is appropriate.
- How to recognize a brute-force pattern by volume and timing, not just by a single failed login.
- Why a success immediately following a burst of failures is a high-priority indicator.
- How correlating two independent log sources (host auth log and network capture) increases confidence in a finding.
- How to turn an ad hoc search into a scheduled detection.